Annexure A

Dry Run Evaluation

CSA17 Artificial Intelligence · Searching Techniques · Slot A (CSA1712) and Slot D (CSA1711)

Name	Register number	Date
NITHYA SHREE K	192521145	26-10-2026

Question 4 – Uniform-Cost Search (UCS)

Task: Find the cheapest path from S to G using Uniform-Cost Search. Always expand the frontier node with the smallest path cost g(n); break ties alphabetically. If a cheaper path to a node already in the frontier is found, replace it. Stop when G is selected for expansion.

Iteration	Frontier before (node:g)	Node expanded	Frontier after (node:g)
1	S:0	S	A:1, B:4
2	A:1, B:4	A	B:3 (replaces B:4 via A), C:6
3	B:3, C:6	B	C:4 (replaces C:6 via B), D:8
4	C:4, D:8	C	G:7, D:8
5	G:7, D:8	G (goal selected – stop)	—
6	—	—	— (not needed)

Questions:

1. Write the order in which nodes are expanded.

S → A → B → C → G

2. What is the cheapest path and its cost?

S → A → B → C → G, cost = 1 + 2 + 1 + 3 = 7.

(The other route S → A → B → D → G costs 1 + 2 + 5 + 2 = 10.)

3. Why does UCS test for the goal when a node is selected, not when it is generated?

The first path found to a node is not necessarily the cheapest. A node is guaranteed to have its minimum g(n) only when it is the smallest in the frontier and is selected. Testing at generation could return a costlier path, so the goal test is done at selection to keep UCS optimal.

Question 7 – Greedy Best-First Search

Task: Find a path from S to G using Greedy Best-First Search, f(n) = h(n). Edge numbers are step costs; h values are written next to each node. Stop when G is selected.

Graph: S–A (2), A–C (2), C–D (3), D–G (1), S–B (5), B–G (6). h: S=7, A=6, B=4, C=4, D=1, G=0.

Iteration	Frontier before (node, h)	Node expanded	Frontier after
1	S(7)	S	B(4), A(6)
2	B(4), A(6)	B	G(0), A(6)
3	G(0), A(6)	G (goal selected – stop)	—
4	—	—	— (not needed)

Questions:

1. Write the path found and its total cost.

S → B → G, total cost = 5 + 6 = 11.

2. Is this path optimal? Why or why not?

No. The optimal path is S → A → C → D → G with cost 2 + 2 + 3 + 1 = 8, which is cheaper than 11.

Greedy uses only h(n) and ignores the cost already spent, g(n). B looked closer to the goal (h = 4) than A (h = 6), so it took the expensive edge B–G (6) and missed the cheaper route.

Question 11 – Simulated Annealing

Task: Simulated annealing is maximising a value. A better move is always accepted. A worse move is accepted with probability P = e^(ΔE/T), where ΔE = neighbour − current. Accept the worse move if the given random number r < P.

Step	Current	Neighbour	ΔE	T	P = e^(ΔE/T)	r	Accept / Reject
1	10	13	+3	20	1 (better move)	—	Accept
2	13	11	−2	10	e^(−0.2) = 0.819	0.55	Accept (0.55 < 0.819)
3	11	7	−4	5	e^(−0.8) = 0.449	0.62	Reject (0.62 > 0.449)
4	11	15	+4	2	1 (better move)	—	Accept
5	15	12	−3	1	e^(−3) = 0.050	0.3	Reject (0.3 > 0.050)

Questions:

1. Why is a worse move more likely to be accepted at high temperature?
   P = e^(ΔE/T). When T is large, ΔE/T is close to 0, so P is close to e^0 = 1 and even bad moves are usually accepted. This lets the search explore and escape local maxima early on.
   Example: ΔE = −2 gives P = 0.819 at T = 10, but only e^(−2) = 0.135 at T = 1.

2. What happens to simulated annealing as T approaches 0?
   ΔE/T → −∞ for a worse move, so P → 0 and worse moves are never accepted. The algorithm becomes plain hill climbing: it only accepts improvements, stops exploring, and can get stuck at a local maximum.

Question 13 – Genetic Algorithm

Task: A genetic algorithm searches for the string 111111. Fitness = number of 1s. Population P1 = 111001, P2 = 101100, P3 = 100100, P4 = 000010. Roulette-wheel selection with random numbers 0.15, 0.62, 0.35, 0.85. Pair 1st with 2nd (crossover after gene 3) and 3rd with 4th (crossover after gene 2). Mutate gene 6 of the first child.

Individual	String	Fitness	Selection probability	Cumulative probability
P1	111001	4	4/10 = 0.4	0.4
P2	101100	3	3/10 = 0.3	0.7
P3	100100	2	2/10 = 0.2	0.9
P4	000010	1	1/10 = 0.1	1.0

Total fitness = 4 + 3 + 2 + 1 = 10.

Questions:

1. Which parents are selected for the four random numbers?
   0.15 falls in 0 – 0.4 → P1 (111001)
   0.62 falls in 0.4 – 0.7 → P2 (101100)
   0.35 falls in 0 – 0.4 → P1 (111001)
   0.85 falls in 0.7 – 0.9 → P3 (100100)
   Selected parents in order: P1, P2, P1, P3.

2. Write the four children after crossover.
   Pair 1 (P1 × P2, after gene 3): 111|001 × 101|100
   Child 1 = 111 100 = 111100 Child 2 = 101 001 = 101001
   Pair 2 (P1 × P3, after gene 2): 11|1001 × 10|0100
   Child 3 = 11 0100 = 110100 Child 4 = 10 1001 = 101001

3. Write the first child after mutation.
   Child 1 = 111100. Flipping gene 6 (0 → 1) gives 111101.

4. Compare the average fitness before and after one generation.
   Before: (4 + 3 + 2 + 1) / 4 = 10 / 4 = 2.5

After: children are 111101 (5), 101001 (3), 110100 (3), 101001 (3) → (5 + 3 + 3 + 3) / 4 = 14 / 4 = 3.5

Average fitness rises from 2.5 to 3.5, so the population moved closer to 111111. (Without the mutation it would be 13 / 4 = 3.25.)

Question 20 – Alpha–Beta Pruning

Task: Apply alpha–beta pruning to the same tree (Question 19), from left to right. Start with $\alpha = -\infty$ and $\beta = +\infty$ at A. Record α and β at each node and mark every pruned leaf.

Step	Node	α	β	What happens
1	A (MAX)	−∞	+∞	Start at A; visit B first. Bounds shown are local to each node.
2	B (MIN)	−∞	+∞	Visit D, the left child of B.
3	D (MAX)	−∞ → 3	+∞	Read leaves 2, 3. D returns 3; B updates β to 3.
4	E (MAX)	−∞ → 5	3	Read leaf 5. α ≥ β (5 ≥ 3): prune leaf 9. E returns bound 5.
5	B → A	A: −∞ → 3	B: 3	B returns min(3, 5) = 3. A updates α to 3, then visits C.
6	C (MIN)	3	+∞	Visit F with inherited α = 3 and β = +∞.
7	F (MAX)	3	+∞	Read leaves 0, 1. F returns 1; α stays 3 because it is an inherited bound.
8	C → A	3	C: +∞ → 1	At C, β ≤ α (1 ≤ 3): prune G and both leaves 7, 5. C returns bound 1. A = max(3, 1) = 3. Total: 5 leaves examined, 3 pruned (9, 7, 5).

Questions:

1. Which leaves are pruned?
(Needs the leaf values.)

2. How many leaves are examined?
(Needs the leaf values.)

3. Is the root value the same as minimax? Why?
Yes. Alpha–beta only cuts branches that cannot influence the final decision at the root (once a MIN node's value falls to α or below, or a MAX node's value rises to β or above, the remaining children cannot change the result). Pruning therefore saves work but never changes the minimax value or the best move.